Sprint 5 - PHP Mastery: From Fundamentals to Frameworks Report

Prepared by: Aicha Nait Dayaa

Supervised by: Mr. Abdelmajid Bendrif
Mr. Hamza Bahlaouane

Date: February 22, 2026

Contents

1 PHP Syntax, Setup & Tooling . . . 2
- 1.1 Introduction . . . 2
- 1.2 Tooling . . . 2
- 1.3 Publishing the package . . . 4

2 Create Your Own Framework . . . 6
- 2.1 Build a Simple PHP MVC Framework . . . 6
- 2.2 Build a Framework Using Symfony Components . . . 9
- 2.3 Github Repo: . . . 9

3 Working with Databases & CLI . . . 10
- 3.1 Overview . . . 10
- 3.2 Project structure . . . 10
- 3.3 Database Setup . . . 11
- 3.4 The Import Process . . . 12

4 REST API Using Laravel . . . 14
- 4.1 Initializing The Project . . . 14
- 4.2 The model . . . 14
- 4.3 The Controller . . . 15
- 4.4 API Route (routes/api.php) . . . 15
- 4.5 Basic Auth . . . 16
- 4.6 Testing with Postman . . . 16

1 PHP Syntax, Setup & Tooling

1.1 Introduction

The goal of this exercise was to get familiar with PHP syntax, PHPDocs, tooling, and package workflows

https://github.com/a-naitdayaa/dummy_package

1.2 Tooling

After typing the **src/** code of the **PHP League Config** and installing the required dev tools **PHPStan** for static analysis and **PHPCS** for code style, both tools were executed to evaluate the code.

- **PHPStan**

  To identify any potential type inconsistencies and code issues :

```
vendor/bin/phpstan analyse src/
```

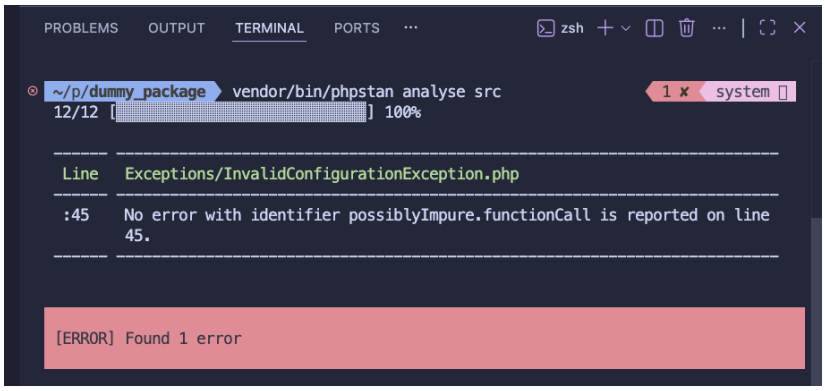


Figure 1: PHPStan Analysis on src/ folder

  We can configure PHPStan to enforce coding standars or analyse code according to specific rulesets, like **PSR-12**

```
vendor/bin/phpstan analyse --standard=PSR-12 src
```

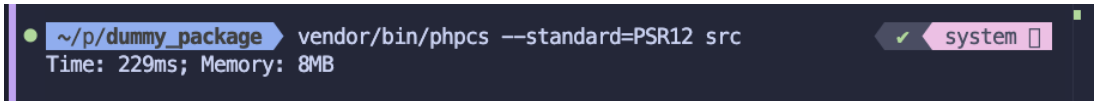
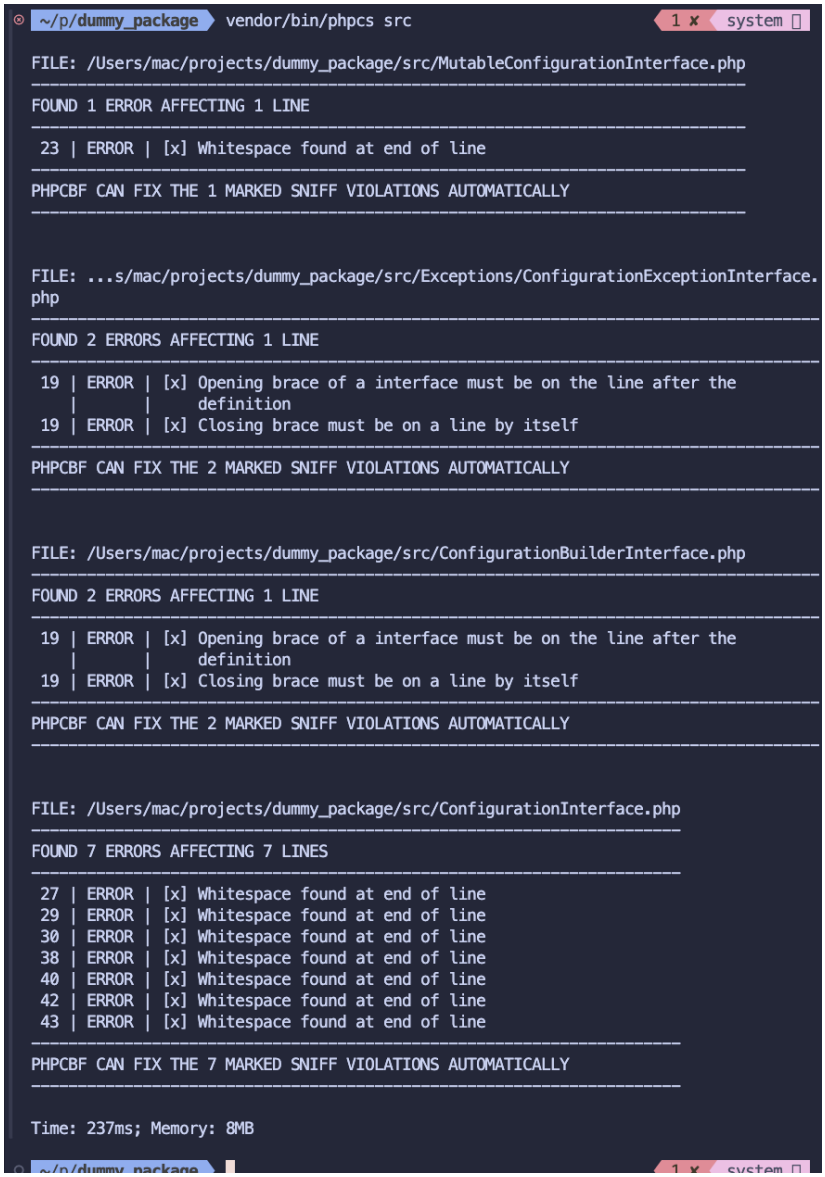


Figure 2: PHPStan Analysis on src/ folder using the standard PSR-12

- **PHPCS**

  To check coding standards :

```
vendor/bin/phpcs src/
```



```
~/p/dummy_package  vendor/bin/phpcs src                    1 ✘  system

FILE: /Users/mac/projects/dummy_package/src/MutableConfigurationInterface.php
----------------------------------------------------------------------------
FOUND 1 ERROR AFFECTING 1 LINE
----------------------------------------------------------------------------
 23 | ERROR | [x] Whitespace found at end of line
----------------------------------------------------------------------------
PHPCBF CAN FIX THE 1 MARKED SNIFF VIOLATIONS AUTOMATICALLY
----------------------------------------------------------------------------


FILE: ...s/mac/projects/dummy_package/src/Exceptions/ConfigurationExceptionInterface.
php
--------------------------------------------------------------------------------
FOUND 2 ERRORS AFFECTING 1 LINE
--------------------------------------------------------------------------------
 19 | ERROR | [x] Opening brace of a interface must be on the line after the
    |       |     definition
 19 | ERROR | [x] Closing brace must be on a line by itself
--------------------------------------------------------------------------------
PHPCBF CAN FIX THE 2 MARKED SNIFF VIOLATIONS AUTOMATICALLY
--------------------------------------------------------------------------------


FILE: /Users/mac/projects/dummy_package/src/ConfigurationBuilderInterface.php
--------------------------------------------------------------------------------
FOUND 2 ERRORS AFFECTING 1 LINE
--------------------------------------------------------------------------------
 19 | ERROR | [x] Opening brace of a interface must be on the line after the
    |       |     definition
 19 | ERROR | [x] Closing brace must be on a line by itself
--------------------------------------------------------------------------------
PHPCBF CAN FIX THE 2 MARKED SNIFF VIOLATIONS AUTOMATICALLY
--------------------------------------------------------------------------------


FILE: /Users/mac/projects/dummy_package/src/ConfigurationInterface.php
----------------------------------------------------------------------
FOUND 7 ERRORS AFFECTING 7 LINES
----------------------------------------------------------------------
 27 | ERROR | [x] Whitespace found at end of line
 29 | ERROR | [x] Whitespace found at end of line
 30 | ERROR | [x] Whitespace found at end of line
 38 | ERROR | [x] Whitespace found at end of line
 40 | ERROR | [x] Whitespace found at end of line
 42 | ERROR | [x] Whitespace found at end of line
 43 | ERROR | [x] Whitespace found at end of line
----------------------------------------------------------------------
PHPCBF CAN FIX THE 7 MARKED SNIFF VIOLATIONS AUTOMATICALLY
----------------------------------------------------------------------

Time: 237ms; Memory: 8MB
```

Figure 3: PHPCS Analysis on src/ folder

These errors are then automatically corrected by the tool **PHPCBF**, the PHP Code Beautifier and Fixer included with **PHPCS**

```
vendor/bin/phpcbf src/
```

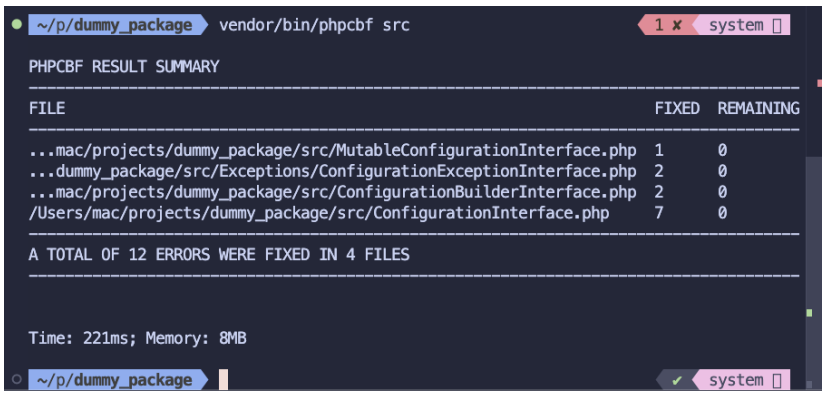


Figure 4: PHPCBF Correction on src/ folder

1.3 Publishing the package

Before publishing a PHP package, certain requirements must be met to ensure it is valid and installable via Composer. These include having a properly configured **composer.json** file with a unique package name, a description, the type, autoloading rules (PSR-4), and any runtime or development dependencies.

A **README** should also be included to provide context. Following these rules ensures that the package can be recognized by Packagist and installed correctly.

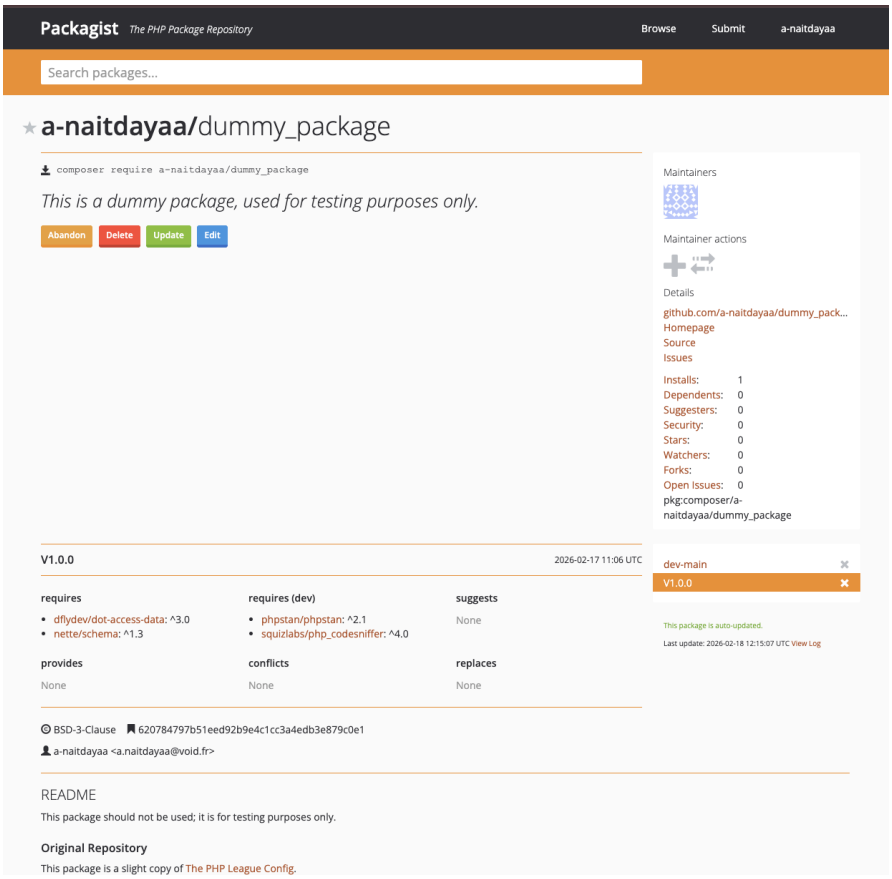


Figure 5: My package on packagist

Then I tested the installation in a different project using Composer

```
~/p/custom_php_framework  main  composer require a-naitdayaa/dummy_package
./composer.json has been updated
Running composer update a-naitdayaa/dummy_package
Loading composer repositories with package information
Updating dependencies
Lock file operations: 4 installs, 0 updates, 0 removals
  - Locking a-naitdayaa/dummy_package (V1.0.0)
  - Locking dflydev/dot-access-data (v3.0.3)
  - Locking nette/schema (v1.3.4)
  - Locking nette/utils (v4.1.3)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 4 installs, 0 updates, 0 removals
  - Downloading a-naitdayaa/dummy_package (V1.0.0)
  - Installing nette/utils (v4.1.3): Extracting archive
  - Installing nette/schema (v1.3.4): Extracting archive
  - Installing dflydev/dot-access-data (v3.0.3): Extracting archive
  - Installing a-naitdayaa/dummy_package (V1.0.0): Extracting archive
Generating autoload files
39 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^1.0 for a-naitdayaa/dummy_package

~/p/custom_php_framework  main !2  ✓  4s  system
```

Figure 6: a-naitdayaa/dummy-package installation

2 Create Your Own Framework

2.1 Build a Simple PHP MVC Framework

This section documents the process of building a simple PHP MVC framework from scratch, following the tutorial written by Anat Garg.

https://github.com/a-naitdayaa/mvc_framework

Project Structure

The framework uses the following directory structure :

```
~/p/mvc_framework  main  tree
.
├── application
│   ├── controllers
│   │   └── Itemscontroller.php
│   ├── models
│   │   └── Item.php
│   └── views
│       └── items
│           ├── add.php
│           ├── delete.php
│           ├── footer.php
│           ├── header.php
│           ├── view.php
│           └── viewall.php
├── config
│   └── config.php
├── db
├── library
│   ├── bootstrap.php
│   ├── controller.class.php
│   ├── model.class.php
│   ├── shared.php
│   ├── sqlquery.class.php
│   └── template.class.php
├── public
│   ├── css
│   ├── img
│   ├── index.php
│   ├── js
│   └── swf
├── scripts
└── tmp
    ├── cache
    ├── logs
    │   └── error.log
    └── sessions

19 directories, 17 files
```

Figure 7: Project structure

- **application/** : Contains the application-specific logic, including controllers, models, and views. This is where the business logic and presentation layer are implemented.
- **config/** : Stores configuration files such as database credentials and environment settings required for the application to run.
- **library/** : Contains the core framework classes that provide the base functionality of the system (e.g., Controller, Model, Template, SQLQuery).

  - **controller.class.php** : Base controller class extended by all application controllers. It coordinates requests, interacts with models, and renders views.
  - **model.class.php** : Base model class extended by application models. It auto detects its own table name from the class and inherits database operations from `SQLQuery`.
  - **sqlquery.class.php** : Provides the database abstraction layer, handling database operations such as querying, inserting, updating, and deleting records.

- **public/** : Web root - the only publicly accessible folder
- **tmp/logs** : Error logs

Apache Configuration

1. **Apache Server Config** :
   Before configuring the application routing through .htaccess, the Apache server requires additional configuration to allow rules rewriting.

   A dedicated VirtualHost was configured for the application to isolate it from other applications and to override the default server configuration.

```
#
<VirtualHost *:8081>
        ServerName localhost
        DocumentRoot "/Users/mac/projects/mvc_framework"
        ErrorLog "/Users/mac/projects/mvc_framework/tmp/logs/error.log"

        <Directory "/Users/mac/projects/mvc_framework">
                Options Indexes FollowSymLinks
                AllowOverride All
                Require all granted
        </Directory>
</VirtualHost>
#
```

Figure 8: VirtualHost Configuration

   - Without setting the directive **AllowOverride** to **ALL**, Apache would ignore the rewrite rules defined in .htaccess.

   - Additionally, the mod_rewrite module was enabled in Apache, as it is responsible for processing URL rewriting rules.

2. **.htaccess Configuration** :
   The framework uses 2 .htaccess files, that work together. The root .htaccess that redirects all incoming traffic to the public folder :

```
        <IfModule mod_rewrite.c>
            RewriteEngine on
            RewriteRule     ^$     public/     [L]
            RewriteRule     (.*)   public/$1     [L]
        </IfModule>
```

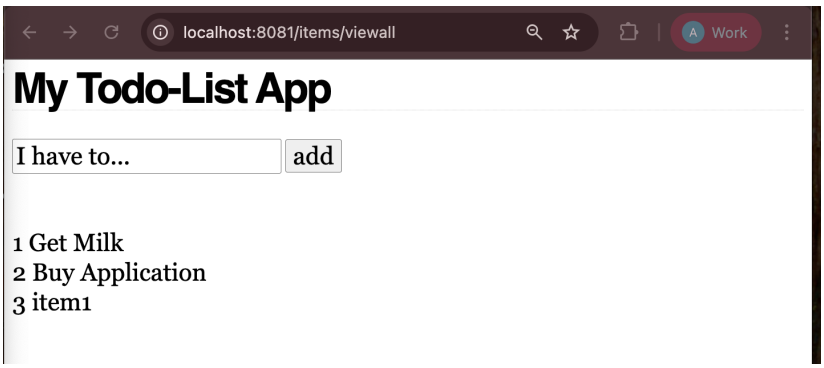
The public/.htaccess then captures all requests and routes them to **index.php**, passing the URL path as the query parameter, whie letting files and directories (images, css, js) pass through directly :

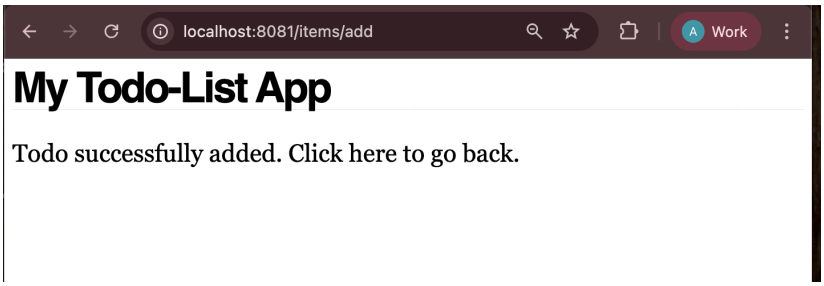
```
        <IfModule mod_rewrite.c>
            RewriteEngine On
            RewriteCond %{REQUEST_FILENAME} !-f
            RewriteCond %{REQUEST_FILENAME} !-d
            RewriteRule ^(.*)$ index.php?url=$1 [PT,L]
        </IfModule>
```

Request Lifecycle

Every request follows a clear lifecycle through the framework:

1. Browser sends a request to **localhost:8081/items/viewall**
2. The root .htaccess redirects it to **public/items/viewall**
3. The public/.htaccess rewrites it to **public/index.php?url=items/viewall**
4. **index.php** then captures the $url, and loads **bootstrap.php**, which in turn loads **config.php** (database credentials) and **shared.php**
5. **shared.php** registers the autoloader, and then calls **CallHook()**
6. **CallHook()** parses the URL, determines the controller and action, derives the model name, and calls the controller method
7. The controller creates the model and template, fetches data from the model, and passes it to the template
8. When the controller is destroyed, **__destruct()** calls **$this→_template→render()**, which includes the header, the action view file (views/items/viewall.php), and the footer.





2.2 Build a Framework Using Symfony Components

2.3 Github Repo:

https://github.com/a-naitdayaa/custom_php_framework

3 Working with Databases & CLI

3.1 Overview

The goal of this exercise was to build a command-line tool that reads a CSV file and loads its contents into a MySQL database.

https://github.com/a-naitdayaa/csv-importer-cli

3.2 Project structure

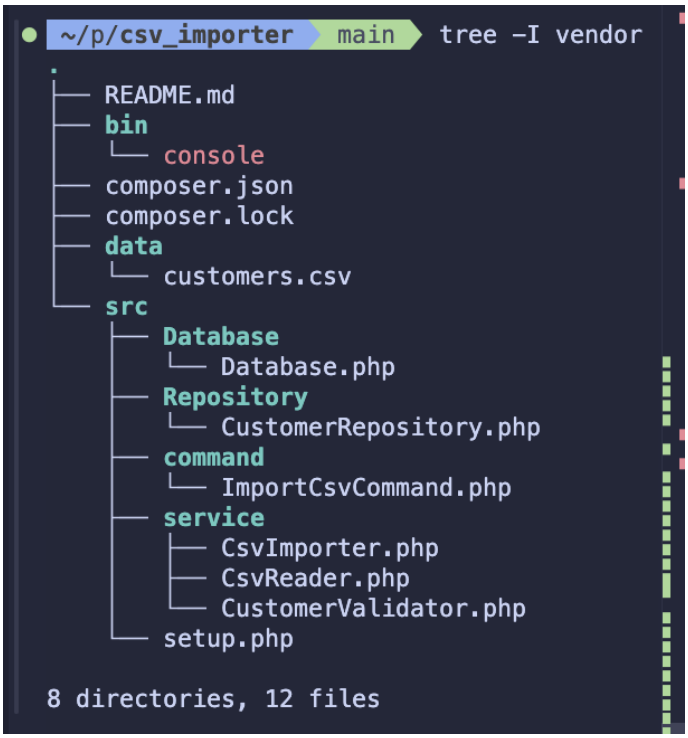


Figure 9: Project Structure

- **bin/console** is The entrypoint, which bootstraps the Symfony Console application and registers the **csv:import** command.
- **setup.php** is a one-time script responsibe for creating the database, the application user, granting the appropriate privileges, and creating the table schema.
- **ImportCsvCommand.php** is the layer that owns the CLI interaction.
- **CsvImporter.php** orchestrates the import process.
- **CsvReader.php** reads the csv file using the League CSV library.
- **CustomerValidator.php** validates row for minimal validation.

- **CustomerRepository.php** is the only component that interacts directly with the database to insert the batches.
- **Database.php** is responsible for instantiating and returning the PDO connection.

3.3 Database Setup

To create the database I implemented a separate PHP script `setup.php` responsible for initializing the database environment.

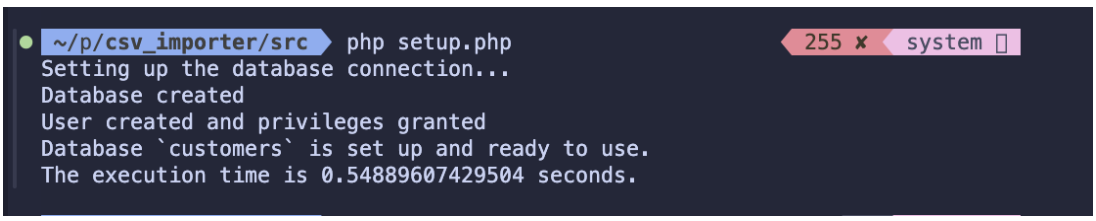
The application user is granted privileges scoped exclusively to the import database:

```
CREATE USER IF NOT EXISTS 'csv_user'@'localhost' IDENTIFIED BY '...';
GRANT ALL PRIVILEGES ON csv_import_db.* TO 'csv_user'@'localhost';
```

This follows the principle of least privilege; the application receives only the access it actually needs and nothing more

Database Schema

```
CREATE TABLE CUSTOMERS (
    id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    customer_id VARCHAR(20) NOT NULL UNIQUE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    company VARCHAR(100) NOT NULL,
    country VARCHAR(100) NOT NULL,
    city VARCHAR(100) NOT NULL,
    Phone1 VARCHAR(30) NOT NULL,
    Phone2 VARCHAR(30) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    subscription_date DATE NOT NULL,
    website VARCHAR(255) NOT NULL )"
);
```



3.4 The Import Process

Initial Approach

In the initial implementation, I adopted a row by row insertion strategy. Each record triggered an individual **INSERT** statement, wich is very inefficient for handling large files.

Figure 10: Execution time before optimization

Bulk Inserting

To improve performance, the solution was to implement a batching mechanism in which records are accumulated and inserted using a single multi-row **INSERT** statement per batch, rather than executing one statement per row.

```
INSERT INTO customers (customer_id, first_name, ...)
VALUES
(?, ?, ...),
(?, ?, ...),
(?, ?, ...);
```

With a batch size of 1000 rows, the number of database round trips was reduced from one per row to one per 1000 rows, which reduced execution time.

Transactions

While bulk insertion improves performance, it introduces a potential consistency issue. If an error occurs during a batch insert, it may not be immediately clear which rows were successfully written to the database and which were not.

To guarantee atomicity of the database, each batch insert is wrapped inside a **beginTransaction()** and **commit()** block. If the insert operation completes successfully, the transaction is committed. If an error occurs, a rollBack() is executed, which cancels the entire batch.

```
~/p/csv_importer  main !1  php bin/cons
ole csv:import data/customers.csv
Importing data from CSV file: data/custome
rs.csv
Import complete!
Rows inserted: 10000
Rows skipped: 0
Time taken: 0.6s
```

Figure 11: Execution time after optimization

4 REST API Using Laravel

This chapter presents the implementation of a RESTful API built using the Laravel framework. The API exposes customer records stored in a MySQL database and provides controlled access through HTTP Basic Authentication.

https://github.com/a-naitdayaa/laravel-rest-api

4.1 Initializing The Project

The project was initialized using composer :

```
composer create-project laravel/laravel .
```

After installation, the project structure includes folders such as **app/**, **routes/**, **bootstrap/**, and **config/**, which follow the MVC architecture.

Once the framework installed, the next step is to configure the database connection. Laravel uses the **.env** file to manage environment variables, such as database credentials.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=customers
DB_USERNAME=user
DB_PASSWORD=pass
```

4.2 The model

```
php artisan make:model Customer
```

In Laravel, models allow interaction with data using Eloquent ORM instead of raw SQL queries.

The Customer model is generated using Artisan:

```
php artisan make:model Customer
```

The model is then configured to match the existing **customers** table:

```
class Customer extends Model
{
    protected $table = 'customers';
    protected $primaryKey = 'id';
    public $timestamps = false;

    protected $fillable = [
        'customer_id',
        'first_name',
```

```
            'last_name',
            'company',
            'country',
            'city',
            'Phone1',
            'Phone2',
            'email',
            'subscription_date',
            'website',
        ];
    }
```

4.3 The Controller

After defining the model, the next step is creating the controller responsible for handling incoming API requests.

The controller was generated using Artisan:

```
php artisan make:controller Api/CustomerController
```

The main logic was implemented inside the `index()` method, which handles GET requests for retrieving customers.

Key Points:

- **Filtering:** Allows clients to filter customers by selected columns. Only whitelisted fields are accepted to prevent invalid queries.
- **Sorting:** Supports sorting results by defined columns in ascending or descending order.
- **Pagination:** Uses page number and page size parameters to return results in chunks, including metadata like current page, total items, and last page.
- **Structured JSON Response:** Returns data, links, and meta information in a clean, consistent format for easy consumption by front-end applications.
- **Custom Header:** Adds `x-api-version: v1` to all responses for API version tracking.
- **Security:** Basic authentication protects the endpoint; credentials are checked against the default `users` table.

4.4 API Route (routes/api.php)

Once the controller is implemented, the next step is exposing the data through an API endpoint.

The route is defined inside `routes/api.php`:

```
    Route::middleware('auth.basic')
    ->get('/customers', [CustomerController::class, 'index']);
```

Then we register the api route in the bootstrap/app.php file :

```
    return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web:      __DIR__ . '/../routes/web.php',
        api:      __DIR__ . '/../routes/api.php',
        commands: __DIR__ . '/../routes/console.php',
    )
    ->create();
```

4.5 Basic Auth

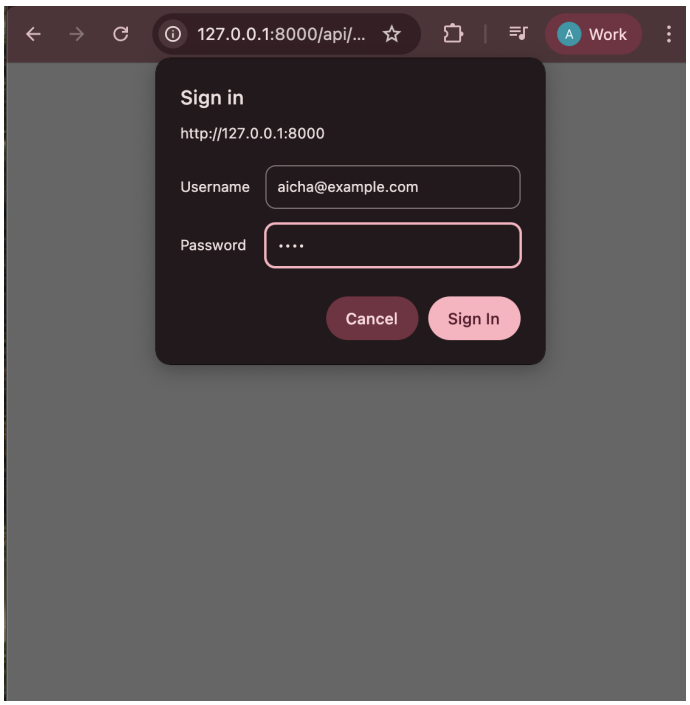
To secure the API endpoints, Laravel's built-in Basic Authentication middleware was used.

Implementation:

```
Route::middleware('auth.basic')
    ->get('/customers', [CustomerController::class, 'index']);
```

The `auth.basic` middleware automatically checks the username and password against the default `users` table.

4.6 Testing with Postman



GET http://127.0.0.1:8000/api/customers?filter[first_name]=Briana&sort[last_name]=asc&page[number]=2&page[size]=10

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Body Cookies Headers (8) Test Results Status: 200 OK Time:

Pretty Raw Preview Visualize JSON


```
{
    "data": [
        {
            "id": 7925,
            "name": "Briana Salas",
            "email": "dorothybates@chan-valdez.net"
        },
        {…
        },
        {…
        },
        {…
        },
        {…
        }
    ],
    "links": {
        "first": "http://127.0.0.1:8000/api/customers?page=1",
        "last": "http://127.0.0.1:8000/api/customers?page=2",
        "prev": "http://127.0.0.1:8000/api/customers?page=1",
        "next": null
    },
    "meta": {
        "current_page": 2,
        "from": 11,
        "last_page": 2,
        "path": "http://127.0.0.1:8000/api/customers",
        "per_page": 10,
        "to": 15,
        "total": 15
    }
}
```


http://127.0.0.1:8000/api/customers

GET http://127.0.0.1:8000/api/customers

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Type Basic Auth

The authorization header will be automatically generated when you send the

Username aicha@example.com

Password 1234

Body Cookies Headers (8) Test Results Status: 200 OK Time: 43

Pretty Raw Preview Visualize JSON


```
{
    "data": [
        {
            "id": 1,
            "name": "Heather Callahan",
            "email": "urangel@espinoza-francis.net"
        },
        {…
        },
        {…
        },
        {…
        },
        {…
        },
        {…
        },
        {…
        },
        {…
        },
        {…
        },
        {…
        },
        {
            "id": 10,
            "name": "Brendan Franco",
            "email": "kentryan@stone-oneill.com"
        }
    ],
    "links": {
        "first": "http://127.0.0.1:8000/api/customers?page=1",
        "last": "http://127.0.0.1:8000/api/customers?page=1000",
        "prev": null,
        "next": "http://127.0.0.1:8000/api/customers?page=2"
    },
```


Console Not connected to a Postman account